T A R E Q O

# 05 — Présentation

La couche `lib/presentation/` contient tout ce que l'utilisateur voit et avec quoi il interagit : **providers** (état Riverpod), **écrans** (par module), **widgets** partagés et **navigation**.

Version 2.1.3+23 · 14 septembre 2026

TAREQO

# Table des matières


État avec Riverpod 3

- Architecture d'état 3
- Providers par module 3

Navigation 4

- `AuthGate` (portail d'authentification) 4
- `AppShell` (coquille adaptative) 4
- Sections (`AppSection`) 4

Écrans 4

- 1. `DashboardScreen` (Accueil) 5
- 2. `ProductListScreen` (Produits) 5
- 3. `SalesScreen` + `NewSaleSheet` + `SaleDetailScreen` (Ventes) 6
- 4. `FinancialScreen` + `AddTransactionSheet` (Finances) 6
- 5b. `CustomersScreen` (Clients) 6
- 5c. `PurchasesScreen` + `AddPurchaseSheet` (Achats) 7
- 5d. `ReportsScreen` (Rapports) 7
- 5. `DebtsScreen` + `AddDebtSheet` (Crédit) 7

Widgets partagés 7

Thème (Material 3) 8

Formatage pt-BR (`Formatters`) 8

Documents connexes 9

# État avec Riverpod

## Architecture d'état

- Providers « pont » (ex. : getProductsUseCaseProvider) lisent depuis GetIt et exposent le cas d'usage comme provider — remplaçables dans les tests via overrideWith/overrideWithValue.
- AsyncNotifiers (AsyncNotifierProvider) exposent l'état comme `AsyncValue` (loading / error / data) — les écrans réagissent de façon déclarative avec .when(...).

## Providers par module

| Fichier | Providers/Notifiers |
|---|---|
| auth_providers.dart | authRepositoryProvider, signIn/signUp/signOutUseCaseProvider, `authStateProvider` (StreamProvider<AppUser?>) |
| products_providers.dart | getProductsUseCaseProvider, addProductUseCaseProvider, `productsProvider` (AsyncNotifier<List<Product>>) |
| sales_providers.dart | salesRepositoryProvider, processSaleUseCaseProvider, `salesListProvider` (trié par date desc) |
| finance_providers.dart | financeRepositoryProvider, getCashFlowUseCaseProvider, addTransactionUseCaseProvider, markTransactionAsPaidUseCaseProvider, `cashFlowPeriodProvider` (enum CashFlowPeriod : aujourd'hui/semaine/mois/tout), `cashFlowProvider` (se reconstruit quand la période change), `transactionsProvider` |
| debts_providers.dart | debtsRepositoryProvider, getPendingDebtsUseCaseProvider, addDebtUseCaseProvider, getSettledDebtsUseCaseProvider, settleDebtUseCaseProvider, `pendingDebtsProvider` (par échéance), `settledDebtsProvider` (par paidAt desc) |
| expiry_alerts_provider.dart | getStockLotsUseCaseProvider, `expiryAlertsProvider` (AsyncNotifier<List<ExpiryAlert>>) — alertes de péremption par lot (W11) |
| dashboard_providers.dart | `dashboardPeriodProvider` (CashFlowPeriod, défaut today) — pilule de période de l'Accueil ; `dashboardCashFlowProvider` (solde de caisse cadencé à la période) ; `dashboardKpisProvider` (RPC get_dashboard_kpis en mode Supabase, repli côté client sinon) |

Détails pertinents :

- cashFlowProvider calcule la période comme suit (tout aligné sur le début de journée) :
- today → début d'aujourd'hui ;
- week → début d'aujourd'hui − 7 jours ;
- month → 1er jour du mois ;
- all → DateTime(2000) jusqu'à maintenant.
- authStateProvider : sans identifiants Supabase, émet Stream.value(null) au lieu d'accéder à Supabase.instance (évite StateError).
- Tous les notifiers ont refresh() (invalidateSelf() + await future).

- Les listes sont copiées avant tri (les entités/listes retournées sont immuables).

---

## Navigation

### AuthGate (portail d'authentification)

lib/presentation/screens/auth/auth_gate.dart décide l'écran racine :

```
authStateProvider.when:
 loading → _SplashScreen (restauration de la session persistée)
 error → LoginScreen
 data → user == null ? LoginScreen : AppShell
```

### AppShell (coquille adaptative)

lib/presentation/screens/shell/app_shell.dart :

- Écrans larges (≥ 880px, web/desktop) : NavigationRail à gauche.
- Écrans étroits (mobile) : NavigationBar en bas.
- Les 5 sections vivent dans un `IndexedStack` (chaque onglet préserve son état : recherche, défilement, formulaires ouverts).
- shellIndexProvider (ShellIndexNotifier) permet de changer d'onglet par programmation — utilisé par les cartes de modules de l'onglet Menu.
- Le FAB central ouvre le menu d'actions global (Nouvelle vente, Enregistrer une dépense, Ajouter un produit, Enregistrer un crédit) — sheets du canvas V3.

### Sections (AppSection)

| Index | Section | Icône | Écran |
|---|---|---|---|
| 0 | Accueil | space_dashboard | DashboardScreen |
| 1 | Produits | inventory_2 | ProductListScreen |
| 2 | Ventes | point_of_sale | SalesScreen |
| 3 | Finances | account_balance_wallet | FinancialScreen |
| 4 | Crédit | receipt_long | DebtsScreen |
| 5 | Clients | people | CustomersScreen |

> Achats et Rapports sont des écrans poussés (Navigator.push) depuis la section « Modules » de l'onglet Menu — ce qui garde la barre à 4 onglets. L'onglet Menu héberge aussi Produits / Crédit / Clients (changement d'onglet), Magasins (propriétaire uniquement) et la tuile Se déconnecter.

---

## Écrans

### 1. DashboardScreen (Accueil)

Vue du jour du canvas V3 — l'onglet Accueil n'a PAS d'avatar (le bouton Se déconnecter vit dans l'onglet Menu) :

- En-tête : date AU-DESSUS de la salutation (« Segunda, 17 ago ») → « Olá, {nom} » ; à droite, la pilule de période (Hoje  : aujourd'hui / semaine / mois / tout — dashboardPeriodProvider cadence les métriques Ventes/Caisse) ; en dessous, le chip de magasin vert foncé (StoreSwitcher, icône storefront blanche + nom ; avec 2+ magasins il ouvre le sélecteur rapide) ;
- 3 cartes de métrique compactes (sans icônes, disposition de l'app de référence) :
- Ventes — ventes de la période sélectionnée dans la pilule avec le delta vs la période précédente en sous-texte coloré (vert +N% / rouge -N%) ;
- Caisse — solde de caisse cadencé à la période (dashboardCashFlowProvider) ; un solde négatif rend la valeur en rouge ; sous-texte = code devise (MZN) ;
- À recevoir — total de crédit en attente avec le compteur de fiados en orange dans le sous-texte (ex. 4 fiados) ;
- Flux d'activité (« Atividade de hoje » + compteur d'événements à droite) dans une carte unique avec icônes circulaires teintées et ancre (valeur/badge) à droite :
- Ventes (icône north_east verte, +valeur en vert) — toucher ouvre SaleDetailScreen ;
- Entrées de stock (icône boîte grise, +qty en blanc) — achats du jour, une ligne par article ;
- Paiements de fiado (icône payments ambre dans un cercle en pointillés, +valeur en vert) — dettes soldées payées aujourd'hui ;
- Dépenses (icône south_west rouge, -valeur en rouge) — dépenses payées du jour ;
- Stock faible (icône d'alerte orange + pilule orange pleine « Alerta ») — produits actifs avec stockQuantity < 5 unités (Quantity.lowStockThresholdUnits) ;
- Péremption (W11) via expiryAlertsProvider : rouges pour les lots périmés avec du stock et jaunes pour les lots expirant dans < 30 jours (expiryWarningDays), triées par urgence (périmés d'abord) ;
- Bannière de synchronisation en attente (outbox du mode SaaS) quand la file hors-ligne n'est pas vide, avec action « Sincronizar » ;
- Une erreur de chargement des KPI affiche un avertissement compact avec réessai (InlineErrorCard) au lieu de faux zéros.

> Le bouton Se déconnecter vit dans l'onglet Menu ; l'en-tête de l'Accueil n'a pas d'avatar. L'Accueil n'a NI actions rapides NI cartes de modules — elles vivent dans le FAB central et l'onglet Menu (même grille de modules et matrice d'accès).

### 2. ProductListScreen (Produits)

- Recherche locale (nom et description), pull-to-refresh, FAB « Nouveau produit » ;
- Cartes avec prix d'achat/vente et chip de stock (jaune si < 5 unités = < 5000 milli-unités) ;

- Inscription via `_AddProductSheet` (bottom sheet) : nom, description, prix, stock, code-barres d'étiquette/PLU (facultatif), interrupteur « Vendu au poids » (vendaPorFracao — active les fractions) et unité de vente (kg/un) — attrape InvalidPriceException et affiche le message ;
- Stock affiché avec fraction et unité pour les produits au poids (ex. 1.75 kg).
- EmptyState pour la liste vide et pour la recherche sans résultats.

## 3. SalesScreen + NewSaleSheet + SaleDetailScreen (Ventes)

- Liste : cartes avec nombre d'articles, moyen de paiement, jour et total ; toucher ouvre le détail ;
- `NewSaleSheet` (panier) :
- Recherche et ajout de produits avec steppers (+/−) limités au stock (soldOut désactive) ; les produits au poids utilisent des pas de 0.1 et ouvrent un dialogue de quantité décimale (ex. 1.75 kg) ;
- Champ code-barres (les scanners style clavier USB déclenchent la recherche sur onSubmitted) + lecture caméra via BarcodeScanSheet (mobile_scanner, là où c'est pris en charge) ;
- Intercepte le motif d'étiquette de balance `20PPPPPWWWWW` (ScaleBarcode) : extrait le PLU, déduit la fraction du pesage (valeur ÷ prix — ne surfacture jamais) et ne traite le code comme un code balance que lorsque le produit est vendaPorFracao — les codes EAN-12 commençant par 20 retombent sur la recherche normale ; les PLU sont normalisés (zéros de tête : 123  00123) ;
- Moyen de paiement via des ChoiceChips (PaymentMethod.values) ;
- Confirme via `ProcessSaleUseCase` → enregistre la vente + déduction de stock FIFO (lots) + entrée en caisse ;
- Attrape InsufficientStockException (affiche le produit et la quantité disponible) et ProductNotFoundException ;
- Après succès, invalide productsProvider, salesListProvider, transactionsProvider, cashFlowProvider, expiryAlertsProvider ;
- `SaleDetailScreen` : résumé (id, date, paiement, total), articles avec quantité × prix et sous-total, et la note « Prix figés au moment de la vente ».

## 4. FinancialScreen + AddTransactionSheet (Finances)

- SegmentedButton de période (CashFlowPeriod) ;
- 3 KPI : Entrées, Sorties, Solde (couleurs sémantiques verte/rouge) ;
- Liste des transactions : icône directionnelle, description, catégorie + jour, valeur signée (+/−) et couleur ;
- Les entrées en attente (entrées avec statut pending) montrent un chip « Pendente » + bouton « Receber » (Recevoir) (dialogue de confirmation → MarkTransactionAsPaidUseCase → invalide caisse/transactions) ;
- `AddTransactionSheet` : type Entrée/Sortie (SegmentedButton), description, catégorie (défaut « Vendas »/« Despesas »), montant et interrupteur « Pendente (fiado) » — attrape InvalidAmountException.

## 5b. CustomersScreen (Clients)

- Recherche locale (nom, téléphone, e-mail), pull-to-refresh et FAB « Nouveau client » ;

- Cartes avec avatar (initiale), nom, contacts et le chip de crédit en attente du client (somme des dettes liées via Debt.customerId) ;
- Inscription via `_AddCustomerSheet` : nom (obligatoire), téléphone et e-mail facultatifs — attrape InvalidCustomerException ;
- Intégration crédit : AddDebtSheet permet de lier la dette à un client (chips) — personName devient le snapshot du nom et customerId garde le lien.

### 5c. PurchasesScreen + AddPurchaseSheet (Achats)

- Liste des achats (fournisseur, nombre d'articles, jour et total) ;
- `AddPurchaseSheet` : fournisseur, recherche de produits avec steppers et total (prix d'achat) ; les produits au poids acceptent les fractions décimales et chaque article peut recevoir une date de péremption (facultative — alimente le lot créé pour l'entrée) → `ProcessPurchaseUseCase` enregistre l'achat, augmente le stock en créant des lots et enregistre la dépense en caisse (catégorie « Compras ») ; attrape ProductNotFoundException ;
- Après succès, invalide achats, produits, transactions, caisse et alertes de péremption.

### 5d. ReportsScreen (Rapports)

- Sélecteur de période (ReportPeriod : Aujourd'hui / 7 jours / Ce mois-ci / Tout) ;
- KPI : Chiffre d'affaires, Bénéfice brut et Marge % ;
- Meilleures ventes (top 5 par quantité, avec chiffre d'affaires) et ventes par jour — calculés par GetSalesReportUseCase à partir des ventes (snapshots) et du catalogue actuel.

### 5. DebtsScreen + AddDebtSheet (Crédit)

- Onglets : « Pendentes » (En attente) et « Quitadas » (Soldées) (SegmentedButton local _DebtView) ;
- En attente : carte résumé « Total à recevoir » (orange) + liste des dettes avec avatar initial, chip d'échéance (En retard rouge / Échoit aujourd'hui jaune / Dans N jours vert) et bouton « Soldé la dette » (dialogue de confirmation → SettleDebtUseCase → invalide crédit, historique, caisse et solde) ;
- Soldées : carte résumé « Total quitado » (Total soldé) (verte) + historique avec paidAt (« Quitada em 25/07/2026 ») ;
- `AddDebtSheet` : nom du client, montant, échéance (DatePicker) → AddDebtUseCase (enregistre la dette + l'entrée en attente liée) ; attrape InvalidDebtAmountException et gère PendingEntryRegistrationException spécialement (informe que le crédit a été enregistré et demande de vérifier Finances — sans suggérer de réessayer).

---

## Widgets partagés

| Widget | Objectif |
|---|---|
| showAppSheet + AppSheetHeader | Bottom sheet standard (coins 28px, SafeArea, padding clavier, titre + fermer) |

| | |
|---|---|
| KpiCard | Indicateur : icône colorée, valeur (avec FittedBox anti-overflow), libellé, légende, état de chargement (« — ») |
| StatusChip | Pilule de statut avec fond translucide et icône facultative |
| EmptyState | État vide : icône dans une boîte arrondie, titre, sous-titre, action facultative |
| ErrorView | État d'erreur : message + détail technique + bouton « Réessayer » |
| SearchField | Champ de recherche rempli, arrondi (sans contour) |
| SectionHeader | Titre de section avec action facultative à droite |
| BarcodeScanSheet | Lecteur de code-barres caméra (mobile_scanner) utilisé au PDV (W11) |

---

## Thème (Material 3)

lib/core/theme/app_theme.dart :

- Seed : vert émeraude profond #0E6B5A ; ColorScheme.fromSeed pour clair/sombre ;
- Couleurs sémantiques (`AppColors`) : income vert #189A5E, expense rouge #D64550, warning orange #E08F2B ;
- Directives visuelles : surfaces neutres, cartes avec bordure subtile sans ombre lourde (elevation: 0), coins arrondis (14–20px), champs remplis sans contour, FilledButtons de 52px de haut ;
- Navigation : NavigationBar (68px de haut, indicateur primaryContainer) et NavigationRail ( groupAlignment: -0.9) ;
- Typographie : échelle modulaire (Apple HIG/SF Pro) : 34 (Large Title — login/bienvenue) → 28 (titres de page) → 24 (KPI/totaux) → 22 (cartes/modales/AppBar) → 20 (sous-titres/sections) → 17/15/14/13/12/11 (corps large/corps/surbrillances de ligne/note/libellés/légende) ; hiérarchie par taille + poids (w300 display, w400 corps, w600+ valeurs), `tabularFigures` sur les styles numériques (les prix MZN et les quantités kg/un s'alignent chiffre à chiffre), et trois axes de précision comme SF Pro : (1) axe optique `opsz` — Inter est variable (14–32) et les titres ≥20px utilisent la coupe *Display* (opsz 24–32) tandis que le corps reste sur la coupe *Text* ; (2) tracking négatif sur les titres (−0.15 à −0.6), neutre sur le corps, légèrement positif sur les légendes ; (3) hauteur de ligne au ratio SF Pro (~1.2 titres, ~1.4 corps) ; la décision complète (tableau d'échelle, rôles et règles d'or) est dans chapitre 12 — Décisions d'architecture ;
- Snackbars flottantes avec thème inversé (sombre sur thème clair et vice versa).

Le thème est appliqué dans app.dart :

```
MaterialApp(
 title: 'TAREQO ERP',
 debugShowCheckedModeBanner: false,
 theme: AppTheme.dark,
 darkTheme: AppTheme.dark,
 themeMode: ThemeMode.dark, // V3 — 100 % sombre
 home: const AuthGate(),
)
```

---

## Formatage pt-BR (Formatters)

lib/core/utils/formatters.dart :

- currency(int cents) → 1.234,56 avec le symbole du tenant (défaut R$, les tenants du Mozambique utilisent MZN) — depuis W4, l'argent est en centimes entiers ;
- date(DateTime) → 04/08/2026 ;
- shortDate(DateTime) → 17 ago (date de l'en-tête du tableau de bord) ;
- dateTime(DateTime) → 04/08/2026 14:30 ;
- day(DateTime) → Hoje, Ontem ou 04/08 ;
- compact(double) → 12,4k (valeurs compactes des métriques) ;
- signedAmount(int cents) → +500,00 / -500,00 (montants du flux).

---

## Documents connexes

- Architecture — comment la présentation parle au domaine.
- Authentification — AuthGate et le flux de session.
- Tests — tests d'écrans avec providers surchargés.